

PROJECT TESTING PHASE

1. API Verification Workflow (Postman / Thunder Client)

API verification ensures that each RESTful endpoint operates correctly according to system specifications, enforces route authorization, and returns standard JSON payloads. Sequential testing is conducted using Postman or the Thunder Client VS Code extension.

A. Authentication Flow

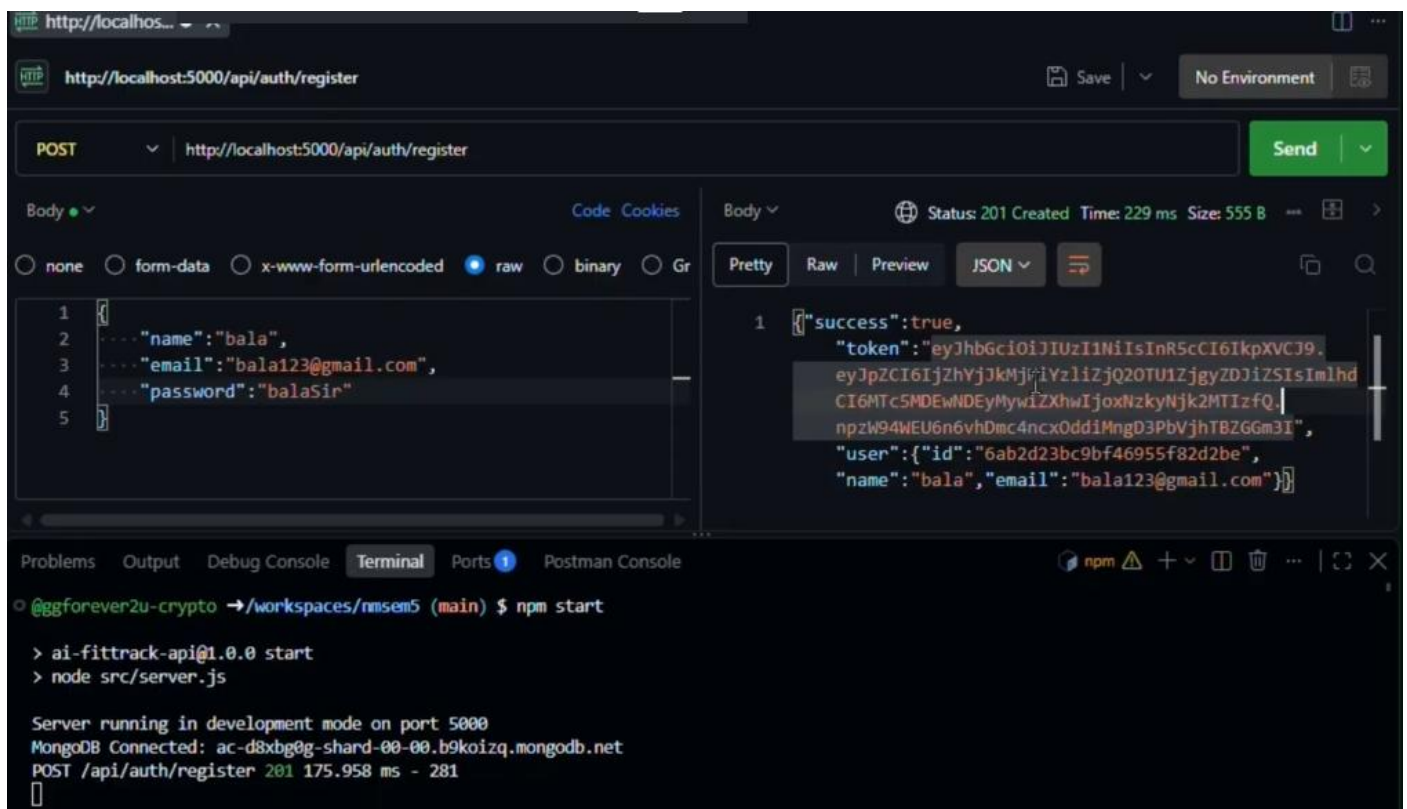
1. POST /api/auth/register

- Purpose: Registers a new user with encrypted password storage.
- Request Payload:

```
{
  "name": "Rahul Sharma",
  "email": "rahul@fittrack.com",
  "password": "Password123"
}
```

- Expected Response (201 Created):

```
{
  "success": true,
  "message": "User registered successfully",
  "data": {
    "_id": "66f41b2a9e3d4c0012a4b891",
    "name": "Rahul Sharma",
    "email": "rahul@fittrack.com"
  }
}
```



2. POST /api/auth/login

- Purpose: Authenticates user credentials and issues a signed JWT bearer token.
- Request Payload:

```
{
  "email": "rahul@fittrack.com",
  "password": "Password123"
}
```

- Expected Response (200 OK):

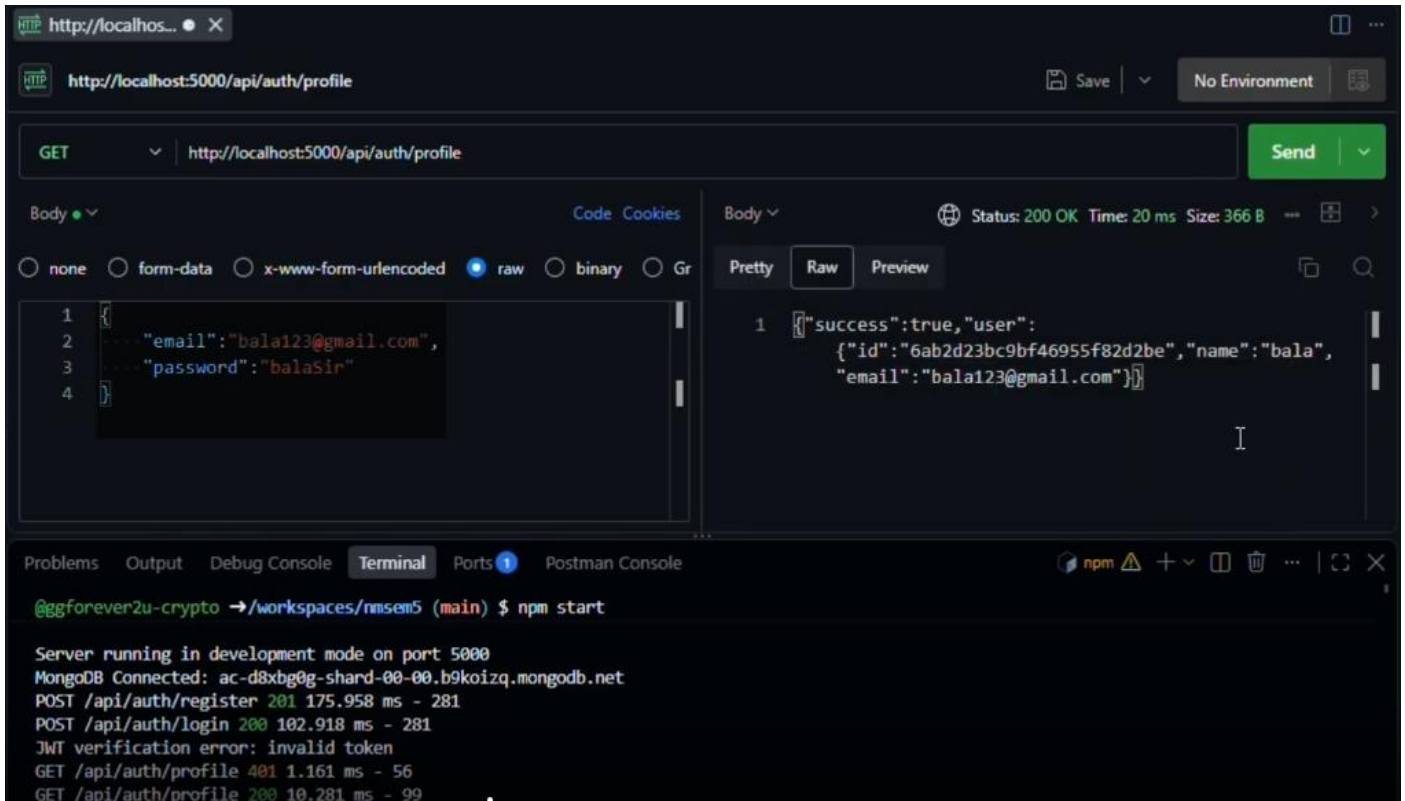
```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

The screenshot displays a Postman interface for a POST request to `http://localhost:5000/api/auth/login`. The request body is a JSON object: `{ "email": "bala123@gmail.com", "password": "balaSir" }`. The response is a 200 OK status with a JSON body: `{ "success": true, "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZhYjJkMjNiYzliZjQ2OTU1ZjgyZDliZSI6ImRhbm94W94WEU6N6vhDmc4ncxOddiMngD3PbVjhTBZGGM3I", "user": { "id": "6ab2d23bc9bf46955f82d2be", "name": "bala", "email": "bala123@gmail.com" } }`. Below the Postman interface, a terminal window shows the command `npm start` being executed, followed by the server starting on port 5000 and logging the successful login request.

3. GET /api/auth/profile

- Purpose: Retrieves authenticated profile details using an Authorization: Bearer <token> header.
- Expected Response (200 OK):

```
{
  "success": true,
  "data": {
    "_id": "66f41b2a9e3d4c0012a4b891",
    "name": "Rahul Sharma",
    "email": "rahul@fittrack.com"
  }
}
```



B. Workout Management Flow

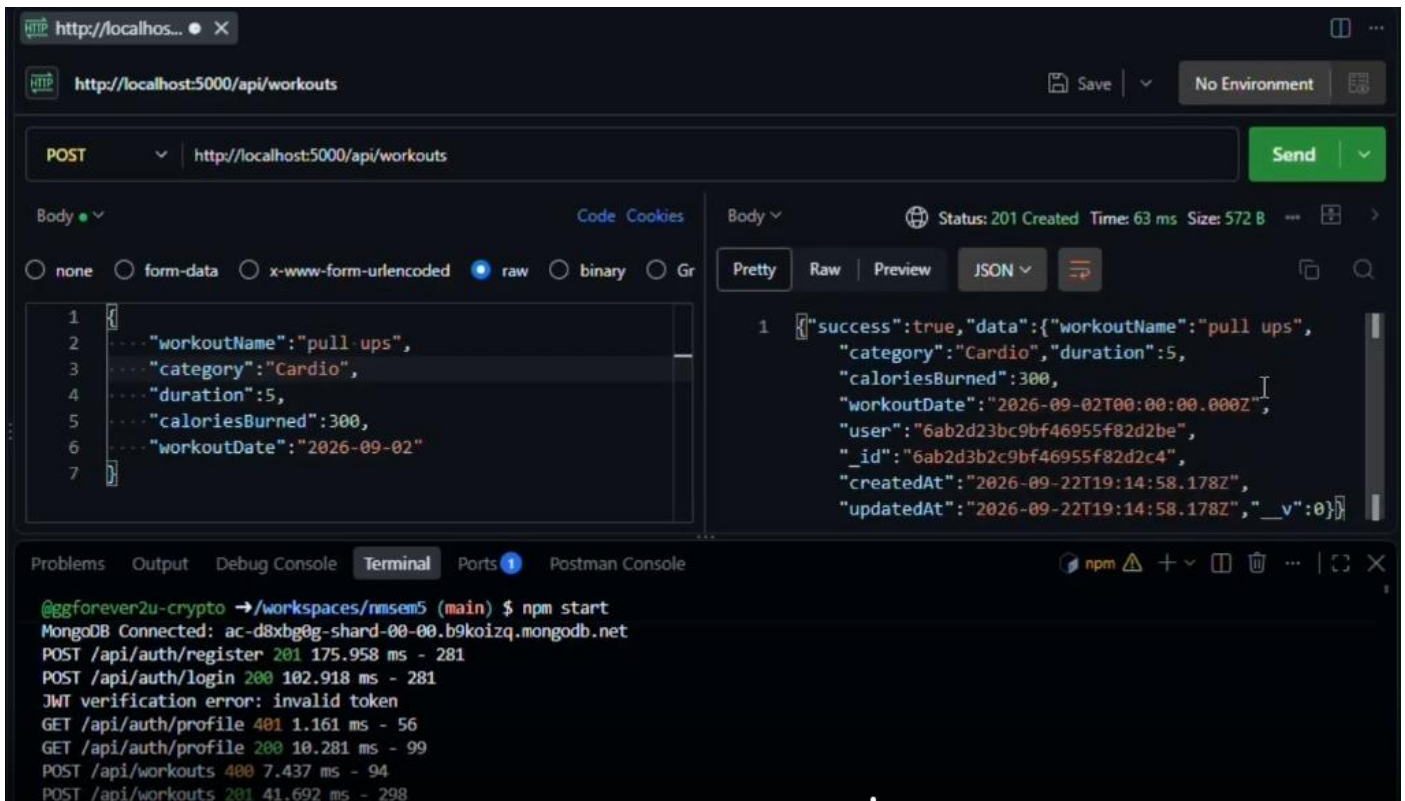
1. POST /api/workouts/

- Purpose: Logs a new workout session for the authenticated user.
- Request Payload:

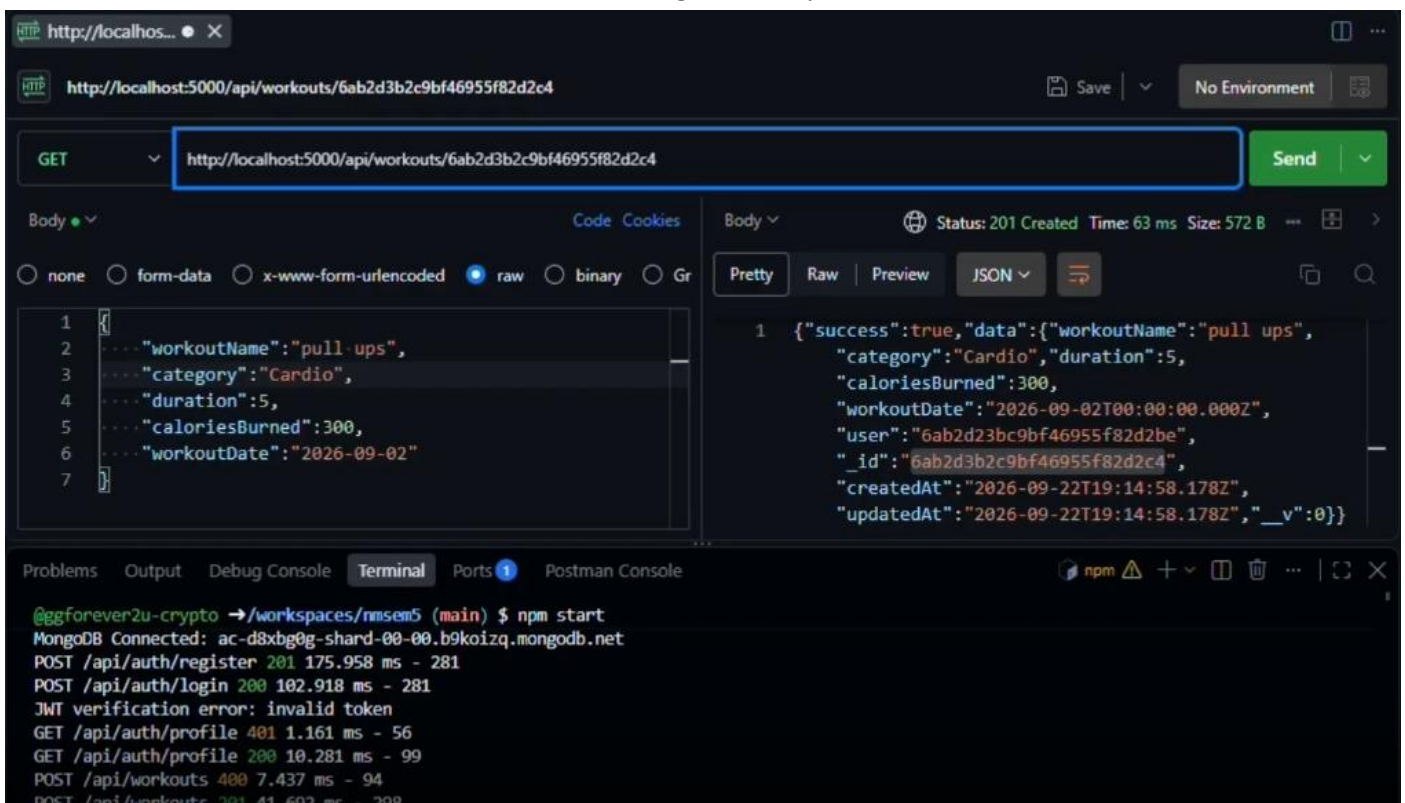
```
{
  "workoutName": "Morning Run",
  "category": "Cardio",
  "duration": 30,
  "caloriesBurned": 250,
  "workoutDate": "2026-09-20"
}
```

- Expected Response (201 Created):

```
{
  "success": true,
  "data": { "_id": "670a...", "workoutName": "Morning Run", "category": "Cardio", "duration": 30,
  "caloriesBurned": 250 }
}
```



2. GET `/api/workouts/search?keyword=run&category=Cardio`
 - Purpose: Filters the logged-in user's workouts by keyword and/or category.
 - Expected Response (200 OK): an array of matching workout documents.
3. PUT `/api/workouts/:id` and DELETE `/api/workouts/:id`
 - Purpose: Updates or permanently removes a specific workout owned by the authenticated user; returns 404 if the record does not belong to the requester.



C. AI Integration Flow

1. POST /api/ai/recommendation

○ Request Payload:

```
{
  "age": 24,
  "fitnessGoal": "Muscle Gain",
  "experienceLevel": "Beginner"
}
```

○ Expected Response (200 OK): { "recommendation": "<AI-generated weekly plan text>" }

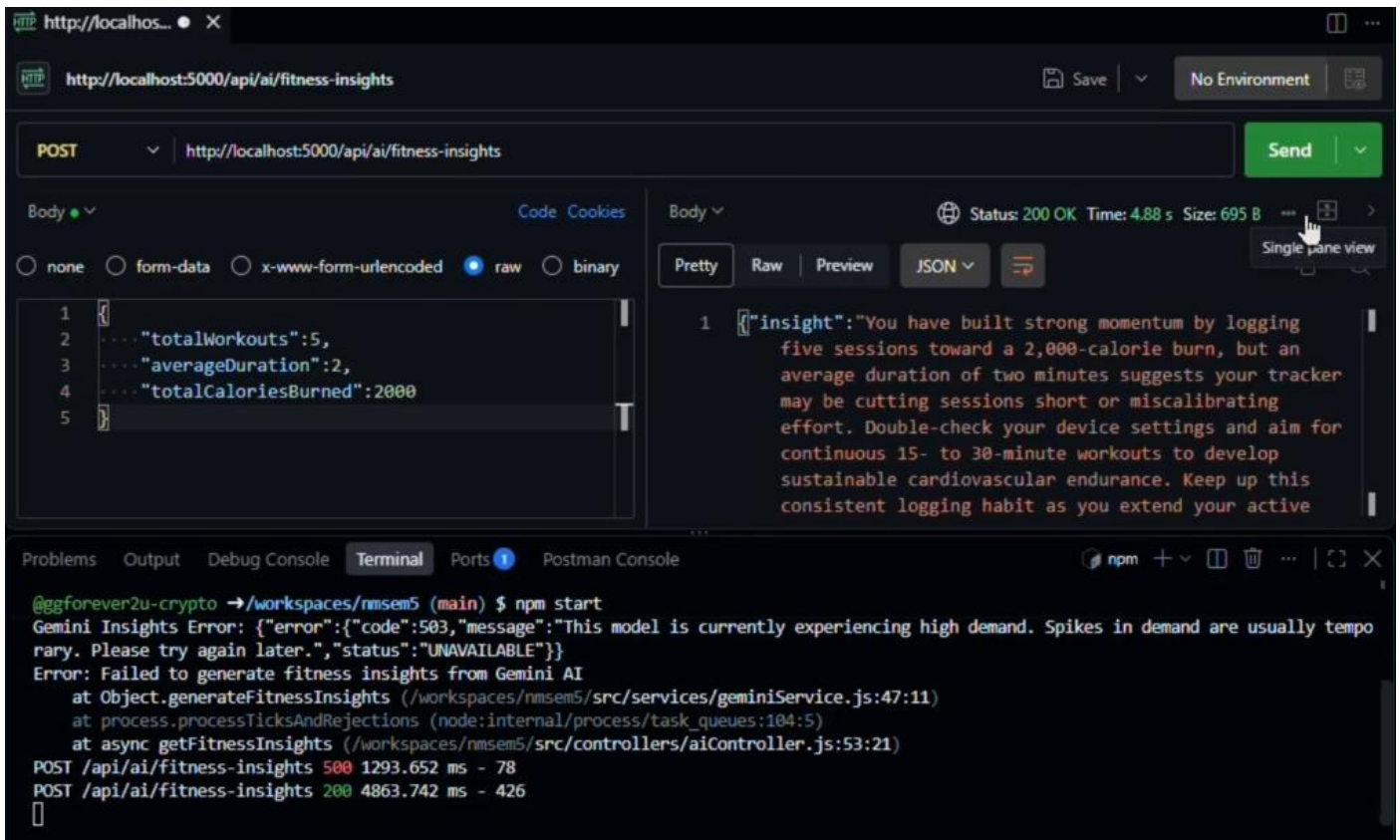
The screenshot shows a web browser with the URL `http://localhost:5000/api/ai/workout-recommendation`. The request method is `POST`. The request body is a JSON object: `{ "age": 24, "fitnessGoal": "Muscle Gain", "experienceLevel": "Beginner" }`. The response status is `200 OK` with a time of `6.33 s` and a size of `1.83 KB`. The response body is a detailed workout plan. Below the browser, a terminal window shows the command `npm start` and the resulting error message: `Gemini Insights Error: { "error": { "code": 503, "message": "This model is currently experiencing high demand. Spikes in demand are usually temporary. Please try again later." }, "status": "UNAVAILABLE" }}`. The terminal also shows the status of the POST requests: `POST /api/ai/fitness-insights 500 6505.517 ms - 78` and `POST /api/ai/workout-recommendation 200 6302.858 ms - 1608`.

2. POST /api/ai/insights

○ Request Payload:

```
{
  "totalWorkouts": 12,
  "averageDuration": 35,
  "totalCaloriesBurned": 3600
}
```

○ Expected Response (200 OK): { "insights": "<AI-generated performance summary>" }



2. Error & Edge Case Verification

- Missing / invalid Authorization header on a protected route returns 401 with a clear message.
- Duplicate email on registration is rejected by the unique index on User.email.
- Requesting or modifying another user's workout by ID returns 404, never another user's data.
- All unhandled exceptions fall through to the centralized error-handling middleware (NFR3), keeping the response shape consistent.